



---

**Lab Number:** 13

---

**Course Code:** CL2005

---

**Course Title:** Database Systems - Lab

---

**Semester:** Spring 2026

---

**Agenda:** Introduction to NoSQL &  
MongoDB

---

**Instructor:** Muhammad Mehdi

---

**Email:** muhammad.mehdi@nu.edu.pk

---

**Office:** Rafaqat Lab

---



## Table of Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>1 Non-Relational Databases (NoSQL).....</b>              | <b>1</b> |
| 1.1 What are Non-Relational Databases?.....                 | 1        |
| 1.2 What is NoSQL?.....                                     | 1        |
| <b>2 Types of NoSQL Databases.....</b>                      | <b>2</b> |
| 2.1 Key-Value Stores.....                                   | 2        |
| 2.2 Document Databases.....                                 | 3        |
| 2.3 Graph Databases.....                                    | 3        |
| 2.4 Column-Store Databases.....                             | 3        |
| <b>3 SQL vs NoSQL Databases.....</b>                        | <b>4</b> |
| 3.1 Key Differences.....                                    | 4        |
| 3.2 When to Use SQL Databases.....                          | 4        |
| 3.3 When to Use NoSQL Databases.....                        | 4        |
| 3.4 Drawbacks of NoSQL.....                                 | 5        |
| <b>4 Introduction to MongoDB.....</b>                       | <b>5</b> |
| 4.1 What is MongoDB?.....                                   | 5        |
| 4.2 The MongoDB Query Language (MQL).....                   | 5        |
| 4.3 Architecture & Components.....                          | 6        |
| 4.4 Key Terms in MongoDB.....                               | 6        |
| <b>5 Getting Started with MongoDB.....</b>                  | <b>6</b> |
| 5.1 Installation & Setup (Windows).....                     | 6        |
| 5.2 Working with MongoDB.....                               | 7        |
| 5.2.1 Viewing Existing Databases.....                       | 7        |
| 5.2.2 Selecting / Creating a Database.....                  | 7        |
| 5.2.3 Viewing Currently Selected Database.....              | 8        |
| 5.2.4 Creating a Collection.....                            | 8        |
| 5.2.5 Viewing Collections.....                              | 8        |
| 5.2.6 Inserting Documents.....                              | 8        |
| 5.2.7 Viewing Documents.....                                | 9        |
| 5.2.8 Counting Documents in a Collection.....               | 10       |
| 5.2.9 Retrieving Distinct Field Values in a Collection..... | 10       |
| 5.2.10 Executing JavaScript Files.....                      | 10       |
| 5.2.11 Dropping a Collection.....                           | 11       |
| 5.2.12 Dropping a Database.....                             | 11       |



# 1 Non-Relational Databases (NoSQL)

## 1.1 What are Non-Relational Databases?

Non-relational databases are database systems that store data in formats other than traditional tables. Instead of rows and columns, they use flexible data models such as:

- **Key-value pairs**
- **Document-based** (JSON-like objects)
- **Graph structures**
- **Wide-column stores**

These databases are designed to handle semi-structured or unstructured data and do not require a fixed schema. This means that records within the same collection can have different structures.

Non-relational databases are particularly useful when:

- Data structure is evolving or not strictly defined
- Applications require rapid development and iteration
- Large volumes of diverse data need to be stored

They prioritize flexibility and performance, making them suitable for modern application development.

## 1.2 What is NoSQL?

NoSQL, short for “**Not Only SQL**,” refers to a class of database systems that do not strictly follow the traditional relational (table-based) model. Instead, they provide flexible ways to store and retrieve data, often using JSON-like structures such as documents, key-value pairs, graphs, or wide-column formats.

NoSQL databases are widely used in modern applications that require handling **large volumes of data**, real-time processing, and distributed architectures. They are designed to **scale horizontally** (across multiple servers), making them well-suited for big data and high-traffic systems.

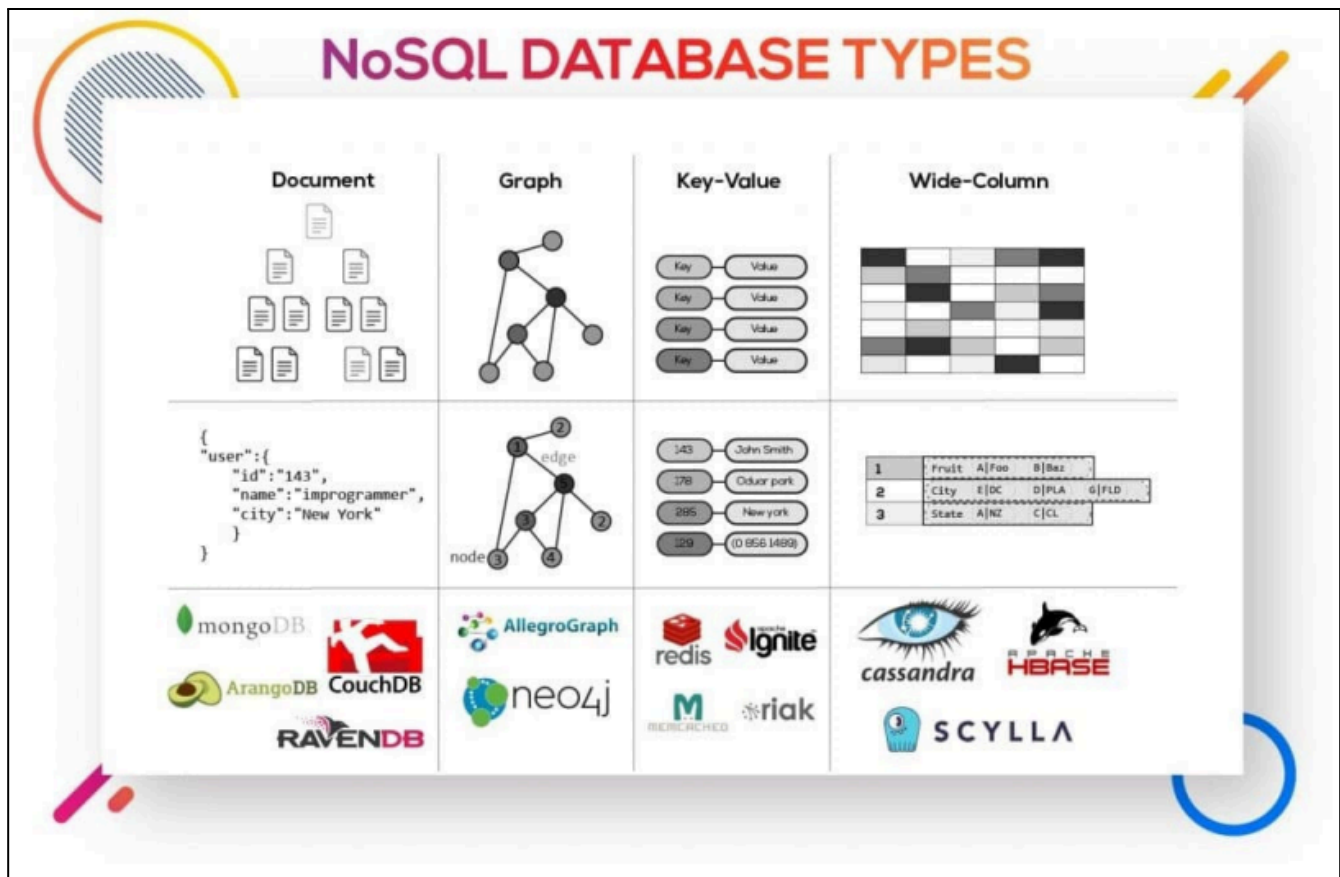
These databases emerged in the late **2000s** to address limitations of traditional relational databases. Their design focuses on:

- High scalability and performance

- Fast data access and queries
- Flexibility to accommodate frequent application changes
- Simpler data models that align closely with application code

In contrast, relational databases, accessed using SQL (Structured Query Language), were developed in the 1970s with an emphasis on minimizing data duplication and maintaining strict data consistency. They use structured, table-based schemas that can be rigid and complex, and they often rely on vertical scaling (upgrading a single server), which can be more costly.

## 2 Types of NoSQL Databases



### 2.1 Key-Value Stores

Key-value databases store data as simple pairs, where each **key** uniquely identifies a corresponding **value**. The value can be anything from a simple string to a complex object.

These databases are optimized for fast read and write operations, making them extremely efficient for simple lookups and high-performance applications such as caching and session management.



**Examples:** Redis, Amazon DynamoDB

## 2.2 Document Databases

Document databases store data in the form of **documents**, typically using JSON or BSON formats. Each document contains structured data in key-value pairs and can include nested fields and arrays.

Unlike relational databases, documents within the same collection can have different structures, providing flexibility for handling evolving data.

These databases are well-suited for applications that require flexible schemas, such as content management systems and web applications.

**Examples:** MongoDB, CouchDB

## 2.3 Graph Databases

Graph databases represent data as **nodes** (entities) and **edges** (relationships between entities). This structure makes it easy to model and query highly connected data.

They are particularly useful for applications such as social networks, recommendation systems, and fraud detection, where relationships between data points are important.

**Examples:** Neo4j, ArangoDB

## 2.4 Column-Store Databases

Column-store (or wide-column) databases organize data into columns rather than rows. This structure allows efficient storage and retrieval of large datasets, especially when only a subset of columns is needed.

They are commonly used in big data and analytics applications where high performance and scalability are required.

**Examples:** Apache Cassandra, HBase



## 3 SQL vs NoSQL Databases

### 3.1 Key Differences

| Feature        | SQL (Relational)                           | NoSQL (Non-Relational)                                  |
|----------------|--------------------------------------------|---------------------------------------------------------|
| Structure      | Tables (rows/columns)                      | Flexible models (documents, key-value, column, graph)   |
| Schema         | Fixed schema                               | Dynamic / schema-less                                   |
| Scaling        | Vertical scaling (upgrade server hardware) | Horizontal scaling (distribute across multiple servers) |
| Query Language | SQL                                        | Varies by database type                                 |
| Relationships  | Strongly defined using joins               | Usually handled through embedding or references         |

### 3.2 When to Use SQL Databases

SQL databases are suitable when:

- Data has a **well-defined and stable structure**
- Applications require **complex relationships and joins**
- **Strong consistency and data integrity** are required
- Used in systems such as:
  - Banking systems
  - Financial applications

### 3.3 When to Use NoSQL Databases

NoSQL databases are preferred when:

- Data structure is **flexible or frequently changing**
- Applications require **large-scale distributed systems**
- High **read/write performance** is needed
- Common use cases include:
  - Real-time applications



- Big data systems
- Social media platforms

## 3.4 Drawbacks of NoSQL

While NoSQL databases offer flexibility and scalability, they have some limitations:

- Lack of a unified standard query language
- Some systems provide weaker consistency models compared to SQL
- Limited support for complex joins and relational queries
- Requires careful data modeling at the application level

# 4 Introduction to MongoDB

## 4.1 What is MongoDB?

MongoDB is a **document-oriented NoSQL database** designed for high-volume data storage and modern application development. It stores data in flexible, JSON-like documents instead of traditional tables and rows. The name MongoDB is derived from the word “**humongous**,” reflecting its ability to handle very large-scale data efficiently.

Documents are the basic unit of data in MongoDB and are stored in **BSON (Binary JSON)** format. Each document consists of **key-value pairs** and can contain nested and flexible structures, allowing for dynamic and evolving data models.

Collections are groups of related documents and can be considered the equivalent of tables in relational databases, although they do not enforce a fixed schema.

MongoDB is designed for **scalability, high performance, and ease of development**, making it widely used in web, mobile, and real-time applications.

## 4.2 The MongoDB Query Language (MQL)

MongoDB uses the **MQL** to interact with databases. MQL is a JSON-like, JavaScript-based syntax used to perform operations such as creating collections, inserting documents, querying data, updating records, and deleting data.



All operations in MongoDB are performed through the **db** object, which represents the currently selected database. Commands are written by chaining the **db** object with collection names and methods.

## 4.3 Architecture & Components

MongoDB follows a Client-Server architecture. It is composed of the following components:

### 1. MongoDB Server:

The database engine that stores and manages data. It runs as a background service.

### 2. Client Tools:

- **Mongo Shell (mongosh):** A command-line interface used to interact with MongoDB using JavaScript-like commands.
- **MongoDB Compass:** A graphical user interface (GUI) tool for visualizing and managing MongoDB data.

## 4.4 Key Terms in MongoDB

| Term       | Description                                                                                            |
|------------|--------------------------------------------------------------------------------------------------------|
| Database   | Container for collections                                                                              |
| Collection | Container for documents (equivalent of a table)                                                        |
| Document   | Basic structure (JSON object) that stores data (equivalent of a row)                                   |
| Field      | Key-value pair within a document (equivalent of a column)                                              |
| <b>_id</b> | Unique identifier for each document (equivalent to a primary key)                                      |
| <b>db</b>  | Represents the currently selected database. It is the primary interface used to perform all operations |

## 5 Getting Started with MongoDB

### 5.1 Installation & Setup (Windows)

MongoDB provides an official installer for Windows that includes the database server and optional tools such as MongoDB Compass (a graphical user interface for managing databases).

**Steps:**





1. Download the [MongoDB Community Server](#) from the official MongoDB website.
2. Run the installer and follow the setup wizard.
3. Choose the Complete installation for all features.
4. Optionally install MongoDB Compass (GUI tool).
5. Install the [MongoDB Shell](#) (mongosh) separately if it is not included in your installer version.
6. Ensure MongoDB is added to your system's PATH (optional but recommended for command-line access).

## 5.2 Working with MongoDB

The typical workflow for working with MongoDB involves starting the database service and connecting to it using a client tool:

- Start the MongoDB service (it may start automatically after installation).
- Open a client tool:
  - **MongoDB Compass** (GUI), or
  - **MongoDB Shell** by opening Command Prompt and running:

```
mongosh
```

- Once connected, you can begin executing commands using MongoDB Query Language (MQL).

### 5.2.1 Viewing Existing Databases

```
show dbs
```

### 5.2.2 Selecting / Creating a Database

**General Syntax:**

```
use database_name
```

**Example:**

```
use lab_13
```



### 5.2.3 Viewing Currently Selected Database

```
db
```

### 5.2.4 Creating a Collection

In MongoDB, all database operations are performed through the **db** object. This object represents the currently selected database and acts as the entry point for interacting with collections, documents, and database-level methods.

Collections are accessed by chaining their names to the **db** object, and actions (such as creating, inserting, or deleting) are performed using methods.

MongoDB allows you to explicitly create a collection using the **createCollection()** method. However, it is important to note that collections are created automatically when you first insert a document into them, so manual creation is optional in many cases.

#### General Syntax:

```
db.createCollection("collection_name")
```

#### Example:

```
db.createCollection("students")
```

### 5.2.5 Viewing Collections

To display all collections in the currently selected database, MongoDB provides a simple shell command.

```
show collections
```

### 5.2.6 Inserting Documents

MongoDB stores data in the form of documents (JSON-like objects). You can insert one or multiple documents into a collection using the following methods.

If the specified collection does not already exist, MongoDB will automatically create it when the first document is inserted.



- `insertOne()` inserts a single document into the collection.
- `insertMany()` inserts multiple documents at once using an array.
- Each document consists of key-value pairs.
- MongoDB automatically adds a unique `_id` field to each document if it is not provided.

#### General Syntax:

```
db.collection_name.insertOne( {key1: value1, key2: value2, ...} )

db.collection_name.insertOne( [ {key: value, ...}, {key: value}, ... ]
)
```

#### Examples:

```
// Insert a single document by passing key-value pairs
db.students.insertOne({
  name: "Ali",
  age: 21,
  cgpa: 3.5
})

// Insert multiple documents by passing an array of documents
db.students.insertMany([
  { name: "Ali", age: 22, cgpa: 3.8 },
  { name: "Ahmed", age: 20 }
])
```

### 5.2.7 Viewing Documents

To retrieve documents from a collection, MongoDB provides the `find()` method.

The `find()` method returns all documents in the specified collection. The result is displayed as a cursor in the MongoDB shell. You can optionally pass query conditions to filter specific documents (covered in later sections).

#### General Syntax:

```
db.collection_name.find()
```

#### Example:



```
db.students.find()
```

### 5.2.8 Counting Documents in a Collection

To find the total number of documents in a collection, MongoDB provides the `countDocuments()` method.

#### General Syntax:

```
db.collection_name.countDocuments()
```

#### Example:

```
db.students.countDocuments()
```

### 5.2.9 Retrieving Distinct Field Values in a Collection

The `distinct()` method is used to get unique values of a specific field in a collection.

#### General Syntax:

```
db.collection_name.distinct('field_name', filter)
```

#### Example:

```
db.students.distinct('department')
```

### 5.2.10 Executing JavaScript Files

MongoDB allows you to execute external JavaScript files containing database commands using the `load()` function.

#### General Syntax:

```
load("file_name.js")
```

#### Example:

```
load("script.js")
```



### 5.2.11 Dropping a Collection

To remove a specific collection from the database, MongoDB provides the `drop()` method.

It deletes the specified collection and all documents within it. The method is called on the collection via the `db` object. This operation cannot be undone.

#### General Syntax:

```
db.collection_name.drop()
```

#### Example:

```
db.students.drop()
```

### 5.2.12 Dropping a Database

MongoDB allows you to delete the currently selected database using the `dropDatabase()` method.

It deletes the currently selected database along with all its collections and documents. This operation is irreversible, so it should be used with caution.

```
db.dropDatabase() // Extremely dangerous
```